

Distributed Lupus in Fabula

Final Report for the Distributed Systems Course 2025/2026

Jacopo Bonifazi Alexandru Nicolescu
author1@email.it author2@gmail.com

Leonardo Vorabbi
leonardo.vorabbi@studio.unibo.it

July 10, 2026

Distributed Lupus in Fabula is a real-time multiplayer web application that brings the classic social deduction game Lupus in Fabula online, allowing groups of players to create or join a game lobby from any browser and play a full match together over the internet. The system is composed of a Vue 3 single-page frontend and a FastAPI/Socket.IO backend, communicating via REST for stateless operations and via WebSocket for real-time, low-latency interactions such as votes, phase transitions, and private role actions. Game and lobby state is kept in Redis rather than a traditional database, acting both as a shared source of truth across backend replicas and as a Pub/Sub broker that lets events generated on one instance reach clients connected to any other instance. This design allows the backend to run as multiple concurrent, interchangeable replicas behind a reverse proxy, supporting many simultaneous matches while keeping every player's view of the game synchronized in real time.

Disclaimer

During the preparation of this work, the authors used LLMs to refine syntax and correct linguistic errors.

After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the final report/artifact

1 Concept

1.1 Type of Product

The project is a web application, specifically a real-time multiplayer game inspired by the traditional social deduction game *Lupus in Fabula*. It is a full-stack distributed system composed of:

- A **frontend**: a Vue 3 Single Page Application (SPA), accessible via browser.
- A **backend**: a set of stateless, horizontally replicated web services built with FastAPI and Socket.IO, responsible for game logic, session management, and real-time event propagation.
- A **shared data layer**: Redis, used both as a low-latency state store and as a Pub/Sub message broker connecting the backend replicas.

The whole system is designed to be deployed as a multi-container distributed application, rather than as a single monolithic server.

1.2 Use Case Description

What the software does. The application lets a group of players create or join a virtual “lobby,” wait until everyone is ready, and then play a phase-based game of hidden roles and social deduction. Once a match starts, the game state moves automatically through a fixed sequence of phases:

LOBBY → DAY → VOTING → NIGHT → ... → ENDED

driven by server-side timers.

During DAY/VOTING, players discuss and vote to eliminate a suspect; during NIGHT, players with special roles perform hidden actions that are resolved by the server before the next day begins. The match ends automatically once a win condition is met.

Where the users are. Players are geographically dispersed (or at least not necessarily co-located): each player connects independently from their own browser, over the public Internet, to a shared game session identified by a room/lobby code.

When and how frequently they interact. Interaction is continuous and real-time for the whole duration of a match (several minutes to tens of minutes), not just at fixed request/response intervals. Players must react within phase time limits (e.g., voting windows, night-action windows), so the system needs low-latency, event-driven communication rather than simple periodic polling.

How they interact. All interaction happens through a web browser via the Vue SPA. The frontend communicates with the backend in two complementary ways:

- **REST** calls for stateless operations such as creating a lobby, joining a lobby, or fetching the current game state.
- **WebSocket** for real-time, bidirectional events: votes, phase changes, chat, private role actions, disconnect/reconnect notifications, etc.

Does the system store user data? Which, and where? The system does not use a persistent relational database for user accounts since there is no long-term profile storage. Instead, it relies on Redis as a shared, TTL-based store for all session-scoped game data:

- room/lobby state and configuration;
- the list of connected players and their ready status;
- assigned roles (Wolf, Seer, Villager);
- votes cast during the DAY/VOTING phase and private night actions;
- phase timers and current phase;
- host identity.

On the client side, the browser itself retains a minimal amount of session data in local storage, purely to support reconnection and continuity of a player's session across page reloads or brief disconnects.

Roles. The system distinguishes several actor roles at different levels:

- *Application-level roles:* **Villager**, **Wolf**, **Seer**, each with different permitted actions and visibility (e.g., Wolves share a private night chat, the Seer can inspect one player per night).
- *Session-level roles:* the **Host**, who configures the match, can remove players from the lobby, and starts the game; versus **regular players**, who can only join, ready up, vote, and act according to their assigned role.
- *System role:* the backend itself acts as an autonomous **orchestrator**. It owns the authoritative game state, advances phases via timers, resolves votes and night actions, and enforces which actions are legal at any given moment.

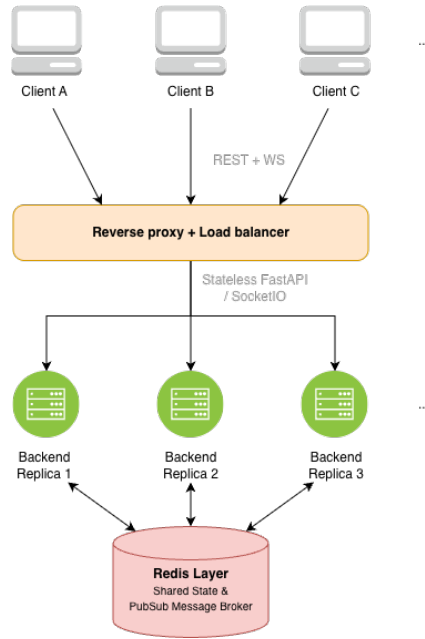


Figure 1: High-level architecture: clients connect through a reverse proxy to stateless backend replicas, which share game state and propagate events through Redis.

1.3 Why Distribution Is Needed

Distribution in this project is not motivated by geographic proximity to users, but primarily by scalability, real-time synchronization, and fault tolerance:

- **Horizontal scalability / resource sharing.** The backend is designed to run as multiple stateless replicas behind a reverse proxy, so that many concurrent lobbies and matches can be served in parallel without a single process becoming a bottleneck. Because game state lives in Redis rather than in each process's memory, any replica can handle requests for any room, allowing the system to scale out simply by adding more backend instances.
- **Shared state across replicas.** Since each backend instance is stateless with respect to game data, Redis acts as the single source of truth for lobby/game state. This is what makes horizontal scaling actually work: a player's WebSocket connection can be handled by one replica while another replica processes a REST call for the same match, and both see a consistent view of the state.
- **Cross-replica real-time propagation.** Because clients belonging to the same room can be connected to different backend replicas, a simple in-process event emitter is not sufficient. The system uses Redis Pub/Sub to broadcast game events (vote updates, phase transitions, chat messages, etc.) across all replicas, ensuring that every client in a room receives updates regardless of which instance generated them or which instance is handling their socket.

- **Fault tolerance and continuity.** The backend includes explicit mechanisms for reconnecting to Redis, retrying failed operations, sweeping/recovering timers, and reconciling state after temporary inconsistencies (anti-entropy). This matters because a real-time game cannot simply crash or stall if a single backend replica fails, a Redis connection drops momentarily, or a player's browser disconnects and reconnects mid-match. The match state and timers must survive these events.
- **Reduced dependency on a single instance.** Because the game state is decoupled from any specific process, a backend replica can be restarted, replaced, or scaled down/up without necessarily terminating ongoing matches, and a reconnecting player can resume their session against a different replica than the one they were originally connected to.

In summary, this system is distributed primarily to support many concurrent matches, keep real-time state synchronized across independent backend replicas, and remain resilient to instance failures and client disconnections.

2 Requirements Elicitation and Analysis

This section specifies *what* the system must do, independently of the specific implementation strategies later adopted. Requirements are grouped into functional, non-functional, and implementation requirements, each with a unique identifier and an associated acceptance criterion to be used during validation.

2.1 Functional Requirements

FR1 – Lobby creation. The system shall allow a user to create a new game lobby by providing a valid nickname.

Acceptance criterion: given a valid nickname, the system creates a room, assigns it a lobby code, and redirects the user to the lobby screen.

FR2 – Joining an existing lobby. The system shall allow a user to join an existing lobby using a room code and a nickname.

Acceptance criterion: with a valid room code, the player enters the lobby and appears in the participant list; with an invalid code, the system displays an error.

FR3 – Browsing open lobbies. The system shall display to the user the list of currently available lobbies.

Acceptance criterion: the home screen shows only lobbies that are still joinable, periodically refreshed from the backend.

FR4 – Host management. The system shall designate a host for each lobby and restrict room-control operations to that host.

Acceptance criterion: only the host can modify lobby settings, remove a player, and start the match.

FR5 – Ready status. The system shall allow players to declare themselves ready or not ready within the lobby.

Acceptance criterion: each player's ready status is updated accordingly, and the match can only start once the minimum required conditions are met.

FR6 – Match start and role assignment. The system shall start a match when requested by the host and assign a role to each participant.

Acceptance criterion: at match start, every player receives a role and the match enters the expected phase sequence.

FR7 – Game phase management. The system shall support the phases LOBBY, DAY, VOTING, NIGHT, and ENDED.

Acceptance criterion: the system displays the current phase and automatically transitions to the next phase according to the game rules.

FR8 – Voting and special actions. The system shall allow players to vote during the day and allow special roles to perform night actions.

Acceptance criterion: votes and actions are accepted only during the correct phase and produce an update visible to the other clients.

FR9 – In-game chat. The system shall allow participants in a room to exchange messages.

Acceptance criterion: a message sent by a user appears to the other players of the same match without a page refresh.

FR10 – Reconnection and session recovery. The system shall allow a disconnected player to reconnect to the same match.

Acceptance criterion: if the client rejoins with the same identity and the same room code, it recovers the correct current match state.

FR11 – Match end and result. The system shall determine the winner and display the final results screen.

Acceptance criterion: when a victory condition is met, the system transitions to ENDED and displays the winning faction, the roles, and the final status of all players.

2.2 Non-Functional Requirements

NFR1 – State consistency. The state of a match shall be consistent across all clients and across all backend replicas.

Acceptance criterion: two clients connected to the same room observe the same match state and the same host, even when served by different backend replicas.

NFR2 – Availability. The system shall remain usable even in the presence of a backend replica failure.

Acceptance criterion: if a backend replica goes down, clients can continue interacting with the match through the remaining active replicas.

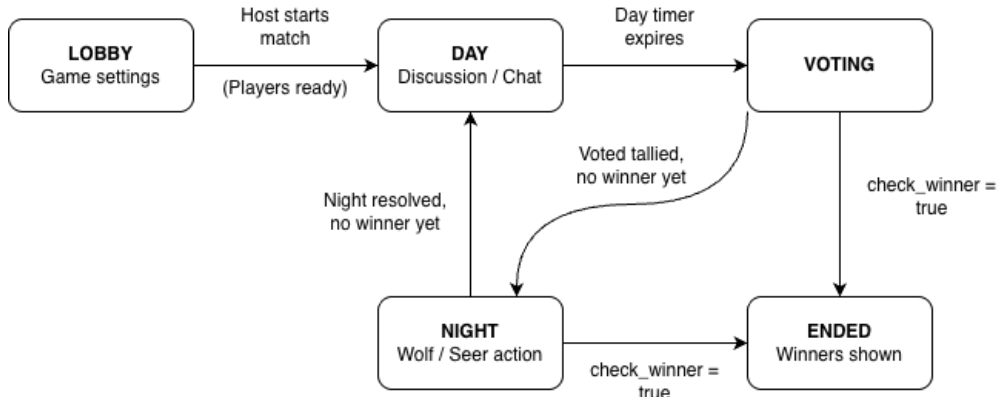


Figure 2: State machine of the match phases. Transitions are driven automatically by server-side timers and by `check_winner()`.

NFR3 – Fault recovery. The system shall recover quickly from temporary disconnections of Redis or of a backend instance.

Acceptance criterion: after a temporary disconnection, the backend re-establishes the connection and clients receive the correct state again without the match being recreated.

NFR4 – Responsiveness. Player actions shall be visible to other participants in real time or near real time.

Acceptance criterion: votes, phase changes, chat messages, and reconnections are propagated without requiring a manual page refresh.

NFR5 – Match data integrity. The system shall not lose the current match state during normal operation.

Acceptance criterion: after restarting a single backend replica, the match state remains recoverable from the shared data store.

NFR6 – Usability. The interface shall allow a non-technical user to join and play a match without needing to understand the system’s internal details.

Acceptance criterion: a user can complete lobby creation/joining, the lobby phase, and the match using only a browser and a room code.

2.3 Implementation Requirements

IR1 – Python backend. The backend shall be implemented in Python using FastAPI and Socket.IO.

Acceptance criterion: the backend service exposes HTTP APIs and WebSocket/Socket.IO channels as specified by the design.

IR2 – Web frontend. The client shall be a Single Page Application built with Vue 3 and Vite.

Acceptance criterion: the application runs in the browser and communicates with the backend via REST and WebSocket.

IR3 – Shared state on Redis. The system shall use Redis as the shared store for lobby state, match state, votes, and timers.

Acceptance criterion: multiple backend replicas read and write the same state without relying on separate local data stores.

IR4 – Containerized deployment. The system shall be executable in containerized environments and support deployment via Docker Compose and Kubernetes.

Acceptance criterion: the full environment starts locally via Compose and can also be deployed on a Kubernetes cluster.

IR5 – Observability. The system shall expose metrics for monitoring purposes.

Acceptance criterion: the backend publishes metrics that can be queried by Prometheus and visualized in Grafana.

These constraints are consistent with the structure already adopted in the project repository: backend, frontend, Redis, reverse proxy, monitoring stack, and Kubernetes manifests.

2.4 Features Less Relevant or Not Central

- **Geographical distribution.** This is not the main motivation for distribution in this project. The system is not designed for users spread across the globe requiring geo-aware routing or replication, but rather to support multiplayer gameplay and backend replication for scalability and fault tolerance.
- **Heterogeneity beyond the standard web stack.** This is not a strong priority. The system's components are built with different technologies (Vue, FastAPI, Redis), but all communicate through standard protocols (HTTP, WebSocket), and no complex integration with heterogeneous third-party systems is required.
- **Strong security model.** Security is not the main focus of the project. The system does not process critical personal data or financial transactions; security concerns are essentially limited to enforcing correct game actions and host privileges.

3 Design

This chapter explains the strategies used to meet the requirements identified in the analysis. Ideally, the design should be the same, regardless of the technological choices made during the implementation phase.

You can re-arrange the sections as you prefer, but all the sections must be present in the end

Important: try to motivate your design choices in relation to the requirements and features identified in section 2 and ??.

3.1 Architecture

- Which architectural style?
 - why?

3.2 Infrastructure

- are there *infrastructural components* that need to be introduced? *how many*?
 - e.g. *clients, servers, load balancers, caches, databases, message brokers, queues, workers, proxies, firewalls, CDNs, etc.*
- how do components *distribute* over the network? *where*?
 - e.g. do servers / brokers / databases / etc. sit on the same machine? on the same network? on the same datacenter? on the same continent?
- how do components *find* each other?
 - how to *name* components?
 - e.g. DNS, *service discovery, load balancing, etc.*

Component diagrams are welcome here

3.3 Modelling

- which **domain entities** are there?
 - e.g. *users, products, orders, etc.*
- how do *domain entities* **map to** *infrastructural components*?
 - e.g. state of a video game on central server, while inputs/representations on clients
 - e.g. where to store messages in an IM app? for how long?
- which **domain events** are there?
 - e.g. *user registered, product added to cart, order placed, etc.*
- which sorts of **messages** are exchanged?
 - e.g. *commands, events, queries, etc.*
- what information does the **state** of the system comprehend
 - e.g. *users' data, products' data, orders' data, etc.*

Class diagram are welcome here

3.4 Interaction

- how do components *communicate*? *when*? *what*?
- *which* **interaction patterns** do they enact?

Sequence diagrams are welcome here

3.5 Behaviour

- how does *each* component **behave** individually (e.g. in *response* to *events* or *messages*)?
 - some components may be *stateful*, others *stateless*
- which components are in charge of updating the **state** of the system? *when*? *how*?

State diagrams are welcome here

3.6 Data and Consistency Issues

- Is there any data that needs to be stored?
 - *what* data? *where*? *why*?
- how should *persistent data* be **stored**?
 - e.g. relations, documents, key-value, graph, etc.
 - *why*?
- Which components perform queries on the database?
 - *when*? *which* queries? *why*?
 - concurrent read? concurrent write? *why*?
- Is there any data that needs to be shared between components?
 - *why*? *what* data?

3.7 Fault-Tolerance

- Is there any form of data **replication** / federation / sharing?
 - *why*? *how* does it work?
- Is there any **heart-beating**, **timeout**, **retry mechanism**?
 - *why*? *among* which components? *how* does it work?
- Is there any form of **error handling**?
 - *what* happens when a component fails? *why*? *how*?

3.8 Availability

- Is there any **caching** mechanism?
 - *where? why?*
- Is there any form of **load balancing**?
 - *where? why?*
- In case of **network partitioning**, how does the system behave?
 - *why? how?*

3.9 Security

- Is there any form of **authentication**?
 - *where? why?*
- Is there any form of **authorization**?
 - which sort of *access control*?
 - which sorts of users / *roles*? which *access rights*?
- Are **cryptographic schemas** being used?
 - e.g. token verification,
 - e.g. data encryption, etc.

4 Implementation

Please report here all the implementation (technology-dependent) choices you made while implementing your design.

If you run out of time for this project, you may consider leaving some aspect unimplemented, and simply discuss how you would have implemented them. In this case, better would be to discuss unimplemented features in section 9.

- which **network protocols** to use?
 - e.g. UDP, TCP, HTTP, WebSockets, gRPC, XMPP, AMQP, MQTT, etc.
- how should *in-transit data* be **represented**?
 - e.g. JSON, XML, YAML, Protocol Buffers, etc.
- how should *databases* be **queried**?
 - e.g. SQL, NoSQL, etc.
- how should components be *authenticated*?

- e.g. OAuth, JWT, etc.
- how should components be *authorized*?
 - e.g. RBAC, ABAC, etc.

4.1 Technological details

- any particular *framework* / *technology* being exploited goes here

5 Validation

5.1 Automatic Testing

- how were individual components **unit-tested**?
- how was communication, interaction, and/or integration among components tested?
- how to **end-to-end-test** the system?
 - e.g. production vs. test environment
- for each test specify:
 - rationale of individual tests
 - how were the test automated
 - how to run them
 - which requirement they are testing, if any

recall that *deployment automation* is commonly used to *test* the system in *production-like* environment

recall to test corner cases (crashes, errors, etc.)

5.2 Acceptance test

- did you perform any *manual* testing?
 - what did you test?
 - why wasn't it automatic?

6 Deployment

- should one install your software from scratch, how to do it?
 - provide instructions
 - provide expected outcomes

- what software should be installed on the machines to run your project? which versions?
- should one set environment variables or configuration files?
- if you're using containerization (e.g. Docker), describe how you engineered your deployment scripts (e.g. `docker-compose.yml` files)

7 User Guide

- how to use your software?
 - provide instructions
 - provide expected outcomes
 - provide screenshots if possible

8 Self-evaluation

- An individual section is required for each member of the group
- Each member must self-evaluate their work, listing the strengths and weaknesses of the product
- Each member must describe their role within the group as objectively as possible.

It should be noted that each student is only responsible for their own section.

9 Future works

Discuss possible future works, improvements, extensions, optimizations, etc.

Also discuss here any unimplemented feature, and how you would have implemented it.

References